

Gestione degli Errori in Java

In Java, la gestione degli errori avviene principalmente attraverso il meccanismo delle eccezioni, un sistema strutturato che permette di gestire situazioni anomale durante l'esecuzione del programma senza interromperne bruscamente il flusso. Questo approccio separa il codice principale dalla gestione degli errori, migliorando leggibilità e manutenibilità.

Tipi di Eccezioni

Java organizza le eccezioni in una gerarchia di classi derivanti da Throwable. Le due sottoclassi principali sono:

- **Error:** rappresenta errori gravi del sistema (es. OutOfMemoryError) che generalmente non devono essere gestiti.
- **Exception:** rappresenta condizioni eccezionali che un'applicazione può voler gestire. Si dividono in:
 - Eccezioni controllate (checked): devono essere dichiarate con throws o gestite con try-catch (es. IOException).
 - Eccezioni non controllate (unchecked): includono RuntimeException e sue sottoclassi (es. NullPointerException, ArrayIndexOutOfBoundsException), non obbligatorie da gestire.

Blocco try-catch-finally

Il costrutto principale per la gestione delle eccezioni è try – catch – finally :

- ➔ Il blocco try contiene il codice a rischio.
- ➔ I blocchi catch gestiscono eccezioni specifiche (ordinati dal più specifico al più generico).
- ➔ Il blocco finally è opzionale e viene eseguito comunque, utile per operazioni di pulizia.

Cosa succede se inserisco Exception prima di qualunque altra tipologia di Exception

Se inserisci un blocco catch (Exception e) prima di un blocco catch (ArithmeticException e), si verifica un errore di compilazione.

Questo perché Exception è la classe genitore di ArithmeticException (e di quasi tutte le eccezioni). Java richiede che i blocchi catch siano ordinati dal più specifico al più generico.

Perché è un errore?

ArithmeticException è un tipo di Exception.

Se Exception viene catturata per prima, il blocco successivo ArithmeticException diventa irraggiungibile, perché ogni ArithmeticException verrebbe già gestita dal primo blocco.

Il compilatore Java blocca questa situazione per evitare codice morto.

```
Scanner var_input = new Scanner(System.in);
int var_dividendo = 55, var_divisore;
float var_risultato = -999;
try {
    // Codice che può generare un'eccezione
    var_divisore = Integer.parseInt(var_input.nextLine());
    var_risultato = var_dividendo / var_divisore;
    IO.println("Nella try var_risultato:" + var_risultato + "<");
}
catch (NumberFormatException err) {
    // Gestione dell'eccezione specifica
    IO.println("È necessario inserire un numero!");
}
catch (ArithmeticException err) {
    // Gestione dell'eccezione specifica
    // IO.println("Ho incontrato un errore aritmetico: " + err);
    IO.println("Il divisore non può essere: " + var_divisore);
    var_risultato = -1;
}
catch (Exception err) {
    // Gestione generica
    // (opzionale, meglio evitare se troppo generica)
    IO.println("Ho incontrato un errore: " + err );
}
finally {
    // Blocco eseguito sempre,
    // utile per pulizia (es. chiudere file)
    IO.println("Sono nella finally");
    var_input.close();
}
IO.println("Il risultato della divisione è: " + var_risultato);
```

```
try {
    int risultato = 10 / 0;
} catch (Exception e) {
    System.out.println("Eccezione generica");
} catch (ArithmeticException e) {
    // ERRORE: mai raggiungibile!
    System.out.println("Divisione per zero");
}
```

Cosa succede se uso più di un try – catch

Blocco di istruzioni dentro try si blocca al primo errore, quindi non si esegue ciò che ha sotto. Per gestire più righe di codice in modo che un errore in una parte non blocchi l'esecuzione del resto, si possono usare blocchi try – catch separati. Ogni blocco gestisce un'operazione indipendente, permettendo al programma di proseguire anche se una di esse fallisce.

L'esempio accanto genera come output

```
Errore calcolo 1: divisione per zero
Errore array: indice fuori limite
Errore conversione: formato numero non valido
Esecuzione completata.
```

Quindi con blocchi try-catch separati, ogni errore viene gestito localmente e il programma continua con i blocchi successivi.

```
// Primo blocco try-catch
try {
    int risultato1 = 10 / 0; // Genera ArithmeticException
    IO.println("Risultato 1: " + risultato1);
} catch (ArithmeticException err) {
    IO.println("Errore calcolo 1: divisione per zero");
}
// Il codice continua nonostante l'errore precedente
try {
    int [] array = {1, 2, 3};
    IO.println("Elemento 5: " + array [5]);
    // Genera ArrayIndexOutOfBoundsException
} catch (ArrayIndexOutOfBoundsException err) {
    IO.println("Errore array: indice fuori limite");
}
// Questo blocco si esegue comunque
try {
    String numero = "123a";
    int n = Integer.parseInt(numero);
    // Genera NumberFormatException
    IO.println("Numero: " + n);
} catch (NumberFormatException err) {
    IO.println("Errore conversione: formato numero non valido");
}
IO.println("Esecuzione completata.");
```

InputMismatchException

InputMismatchException è un'eccezione unchecked (sottoclasse di RuntimeException) che si verifica quando si tenta di leggere un tipo di dato specifico (es. int, double) usando la classe Scanner, ma l'input fornito non corrisponde al formato atteso.

Appartiene al pacchetto java.util, quindi va importata esplicitamente: `import java.util.InputMismatchException;`

Esempio Base con Gestione dell'Errore

```
import java.util.InputMismatchException;
import java.util.Scanner;
public class Esempio {
    public static void main(String [] args) {
        Scanner scan = new Scanner (System.in);
        try {
            IO.print("Inserisci un intero: ");
            int valore = scan.nextInt();
            IO.println("Hai inserito: " + valore);
        } catch (InputMismatchException err) {
            IO.println("Errore: hai inserito un"
                + "valore non numerico!");
        } finally {
            scan.close();
        }
    }
}
```

Se l'utente inserisce "venti", viene catturata l'eccezione e stampato il messaggio di errore.

Esempio con Ripetizione Finché l'Input è Valido

```
Scanner scan = new Scanner (System.in);
int numero = 0;
boolean valido = false;

while (!valido) {
    try {
        IO.print("Inserisci un numero intero: ");
        numero = scan.nextInt();
        valido = true; // Esci dal ciclo se l'input è valido
    } catch (InputMismatchException err) {
        IO.println("Errore: inserisci un numero valido.");
        scan.next(); // Pulisce il token non valido
    }
}
IO.println("Numero valido inserito: " + numero);
scan.close();
```

scan.next() nel blocco catch serve a rimuovere il token errato dal buffer, evitando loop infiniti.

Lancio di Eccezioni

È possibile lanciare eccezioni manualmente con `throw` e dichiararle nel metodo con `throws`.

Sintassi: `throw new TipoEccezione("messaggio");`

Errore comune è dimenticare `new` in fase di dichiarazione. Questo genera errore dal compilatore.

Esempio:

```
public void controlloEtà(int età) throws IllegalArgumentException {
    if (età < 0) {
        throw new IllegalArgumentException("L'età non può essere negativa");
    }
    IO.println("Età valida: " + età);
}
```

Se si chiama `setEtà(-5)`, viene lanciata un'eccezione e l'esecuzione si interrompe (a meno che non sia gestita con `try-catch`).

Eccezioni Personalizzate

Per gestire scenari specifici, si possono creare eccezioni personalizzate:

```
public class EtàNonValidaException extends Exception {
    public EtàNonValidaException(String messaggio) {
        super(messaggio);
    }
}
```

Per poi utilizzarla:

```
public void verificaEtà(int età) throws EtàNonValidaException {
    if (età < 0 || età > 150) {
        throw new EtàNonValidaException("Età non valida: " + età);
    }
    IO.println("Età valida: " + età);
}
```

Nota: `throws` dichiara che il metodo può lanciare un'eccezione, che dovrà essere gestita dal chiamante.

Errori comuni

- Lanciare eccezioni non adatte: Usare `RuntimeException` generica invece di una specifica come `IllegalArgumentException`.
- Non gestire l'eccezione lanciata: Se lanci un'eccezione controllata (checked), devi dichiararla con `throws` o gestirla con `try-catch`.

Differenza tra `throw` e `throws`

<code>throw</code>	<code>throws</code>
Lancia un'eccezione dentro un metodo	Dichiarata nella firma del metodo
<code>throw new IOException();</code>	<code>public void leggi() throws IOException</code>
Usato per generare un'eccezione	Usato per annunciare che il metodo può lanciare eccezioni

